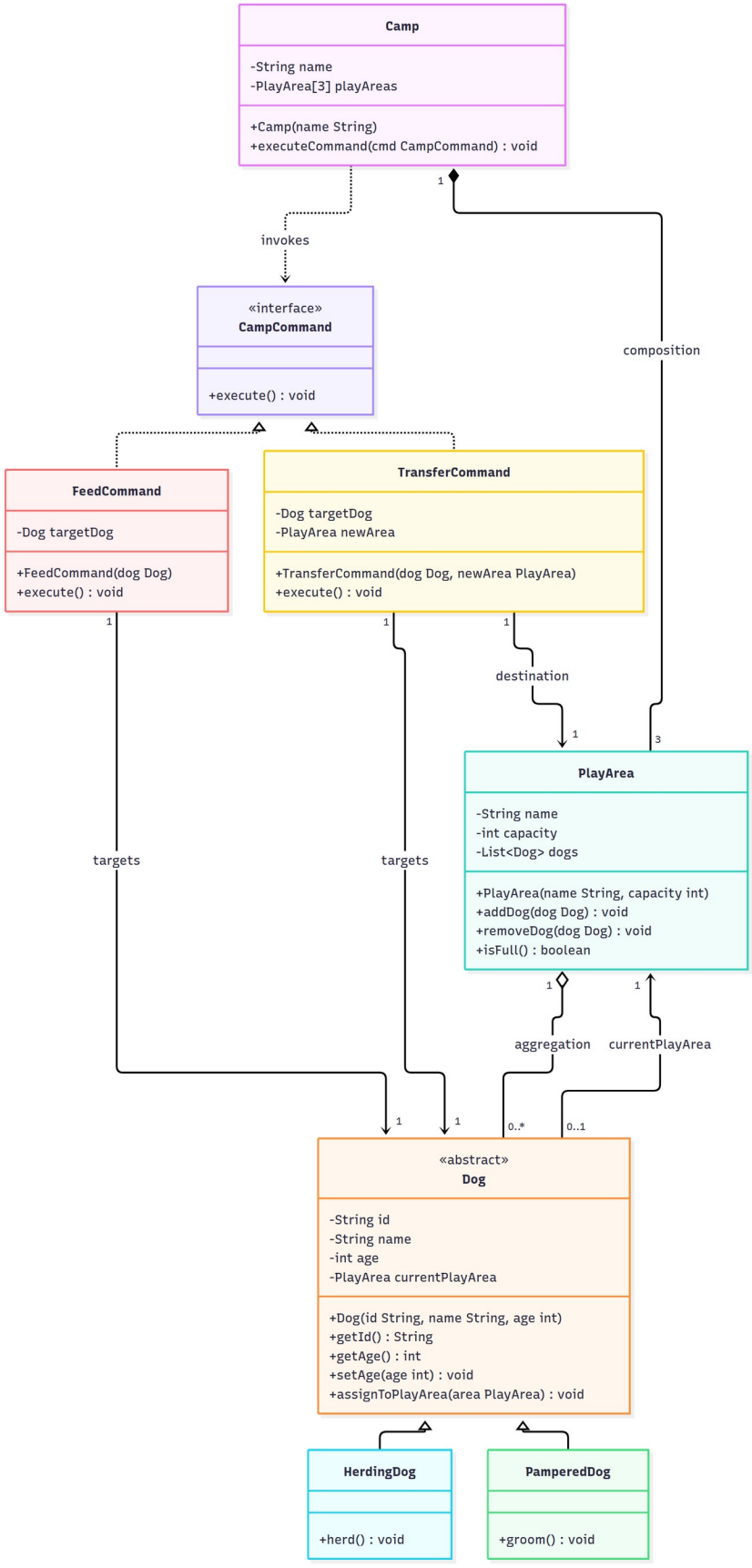
Dog-Sitting Camp Management System

Object-Oriented Design Exercise

Task 1: UML Class Diagram

Camp is composed of exactly three PlayArea instances (composition - filled diamond, multiplicity 3). Each PlayArea aggregates zero or more Dog instances (aggregation - open diamond), and each Dog is currently assigned to at most one PlayArea. Dog is an abstract base class, with HerdingDog and PamperedDog as inheriting subclasses adding herd() and groom(). CampCommand is an interface implemented by FeedCommand and TransferCommand, which target a Dog (and, for TransferCommand, a destination PlayArea). Camp acts as the invoker through executeCommand(CampCommand), depending only on the CampCommand interface rather than concrete command classes (Open-Closed Principle).


Camp
-String name
-PlayArea[3] playAreas
+Camp(name String)
+executeCommand(cmd CampCommand) : void
invokes
composition
1
«interface»
CampCommand
+execute() : void
FeedCommand
-Dog targetDog
+FeedCommand(dog Dog)
+execute() : void
TransferCommand
-Dog targetDog
-PlayArea newArea
+TransferCommand(dog Dog, newArea PlayArea)
+execute() : void
destination
3
PlayArea
-String name
-int capacity
-List<Dog> dogs
+PlayArea(name String, capacity int)
+addDog(dog Dog) : void
+removeDog(dog Dog) : void
+isFull() : boolean
targets
targets
aggregation
currentPlayArea
0..*
0..1
«abstract»
Dog
-String id
-String name
-int age
-PlayArea currentPlayArea
+Dog(id String, name String, age int)
+getId() : String
+getAge() : int
+setAge(age int) : void
+assignToPlayArea(area PlayArea) : void
HerdingDog
+herd() : void
PamperedDog
+groom() : void

Task 2: Java Class Skeletons

Constructor logic and encapsulation validation (immutable id, strictly-positive age) are implemented. The two command classes leave their execute() body as a placeholder, per the task instructions.

Dog.java

```
package campsystem;

public abstract class Dog {

    private final String id;
    private String name;
    private int age;
    private PlayArea currentPlayArea;

    public Dog(String id, String name, int age) {
        this.id = id;
        this.name = name;
        setAge(age);
    }

    public String getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age <= 0) {
            throw new IllegalArgumentException("Age must be strictly positive");
        }
        this.age = age;
    }

    public PlayArea getCurrentPlayArea() {
        return currentPlayArea;
    }

    public void assignToPlayArea(PlayArea area) {
        this.currentPlayArea = area;
    }
}
```

HerdingDog.java

```
package campsystem;

public class HerdingDog extends Dog {

    public HerdingDog(String id, String name, int age) {
        super(id, name, age);
    }
```

```
    public void herd() {
    }
}
```

PamperedDog.java

```
package campsystem;

public class PamperedDog extends Dog {

    public PamperedDog(String id, String name, int age) {
        super(id, name, age);
    }

    public void groom() {
    }
}
```

PlayArea.java

```
package campsystem;

import java.util.ArrayList;
import java.util.List;

public class PlayArea {

    private final String name;
    private final int capacity;
    private final List<Dog> dogs = new ArrayList<>();

    public PlayArea(String name, int capacity) {
        this.name = name;
        this.capacity = capacity;
    }

    public String getName() {
        return name;
    }

    public int getCapacity() {
        return capacity;
    }

    public void addDog(Dog dog) {
        if (isFull()) {
            throw new IllegalStateException("PlayArea is at full capacity");
        }
        dogs.add(dog);
        dog.assignToPlayArea(this);
    }

    public void removeDog(Dog dog) {
        dogs.remove(dog);
        dog.assignToPlayArea(null);
    }
```

```
    public boolean isFull() {
        return dogs.size() >= capacity;
    }
}
```

CampCommand.java

```
package campsystem;

public interface CampCommand {
    void execute();
}
```

FeedCommand.java

```
package campsystem;

public class FeedCommand implements CampCommand {

    private final Dog targetDog;

    public FeedCommand(Dog dog) {
        this.targetDog = dog;
    }

    @Override
    public void execute() {
        // Implementation here
    }
}
```

TransferCommand.java

```
package campsystem;

public class TransferCommand implements CampCommand {

    private final Dog targetDog;
    private final PlayArea newArea;

    public TransferCommand(Dog dog, PlayArea newArea) {
        this.targetDog = dog;
        this.newArea = newArea;
    }

    @Override
    public void execute() {
        // Implementation here
    }
}
```

Camp.java

```
package campsystem;
```

```
public class Camp {

    private final String name;
    private final PlayArea[] playAreas = new PlayArea[3];

    public Camp(String name, PlayArea area1, PlayArea area2, PlayArea area3) {
        this.name = name;
        playAreas[0] = area1;
        playAreas[1] = area2;
        playAreas[2] = area3;
    }

    public String getName() {
        return name;
    }

    public PlayArea[] getPlayAreas() {
        return playAreas;
    }

    public void executeCommand(CampCommand cmd) {
        cmd.execute();
    }
}
```